

Iterator

迭代器

在 STL 中，迭代器（Iterator）用来访问和检查 STL 容器中元素的对象，它的行为模式和指针类似，但是它封装了一些有效性检查，并且提供了统一的访问格式。类似的概念在其他很多高级语言中都存在，如 Python 的 `__iter__` 函数，C# 的 `IEnumerator`。

基础使用

迭代器听起来比较晦涩，其实迭代器本身可以看作一个数据指针。迭代器主要支持两个运算符：自增 (`++`) 和解引用（单目 `*` 运算符），其中自增用来移动迭代器，解引用可以获取或修改它指向的元素。

指向某个 [STL 容器](#) `container` 中元素的迭代器的类型一般为 `container::iterator`。

迭代器可以用来遍历容器，例如，下面两个 `for` 循环的效果是一样的：

```
1 vector<int> data(10);
2
3 for (int i = 0; i < data.size(); i++)
4     cout << data[i] << endl; // 使用下标访问元素
5
6 for (vector<int>::iterator iter = data.begin(); iter != data.end(); iter++)
7     cout << *iter << endl; // 使用迭代器访问元素
8 // 在C++11后可以使用 auto iter = data.begin() 来简化上述代码
```

▼ `auto` 在竞赛中的使用

大部分选手都喜欢使用 `auto` 来代替繁琐的迭代器声明。根据 2021 年 9 月发布的 [关于 NOI 系列活动中编程语言使用限制的补充说明](#)，NOI 系列比赛（包括 CSP J/S）在评测时将使用 **C++14**，这个版本已经支持了 `auto` 关键字。

分类

在 STL 的定义中，迭代器根据其支持的操作依次分为以下几类：

- `InputIterator`（输入迭代器）：只要求支持拷贝、自增和解引访问。
- `OutputIterator`（输出迭代器）：只要求支持拷贝、自增和解引赋值。
- `ForwardIterator`（向前迭代器）：同时满足 `InputIterator` 和 `OutputIterator` 的要求。
- `BidirectionalIterator`（双向迭代器）：在 `ForwardIterator` 的基础上支持自减（即反向访问）。
- `RandomAccessIterator`（随机访问迭代器）：在 `BidirectionalIterator` 的基础上支持加减运算和比较运算（即随机访问）。
- `ContiguousIterator`（连续迭代器）：在 `RandomAccessIterator` 的基础上要求对可解引用的迭代器 $a + n$ 满足 $*(a + n)$ 与 $*(std::address_of(*a) + n)$ 等价（即连续存储，其中 a 为连续迭代器、 n 为整型值）。

`ContiguousIterator` 于 C++17 中正式引入。

▼ 为什么输入迭代器叫输入迭代器？

「输入」指的是「可以从迭代器中获取输入」，而「输出」指的是「可以输出到迭代器」。

「输入」和「输出」的施动者是程序的其它部分，而不是迭代器自身。

其实这个「分类」并不互斥——一个「类别」是可以包含另一个「类别」的。例如，在要求使用向前迭代器的地方，同

样可以使用双向迭代器。

不同的 [STL 容器](#) 支持的迭代器类型不同，在使用时需要留意。

指针满足随机访问迭代器的所有要求，可以当作随机访问迭代器使用。

相关函数

很多 [STL 函数](#) 都使用迭代器作为参数。

可以使用 `std::advance(it, n)` 将迭代器 `it` 向后移动 `n` 步；若 `n` 为负数，则对应向前移动，此时迭代器必须满足双向迭代器，否则行为未定义。

在 C++11 以后可以使用 `std::next(it)` 获得向前迭代器 `it` 的后继（此时迭代器 `it` 不变），`std::next(it, n)` 获得向前迭代器 `it` 的第 `n` 个后继。

在 C++11 以后可以使用 `std::prev(it)` 获得双向迭代器 `it` 的前驱（此时迭代器 `it` 不变），`std::prev(it, n)` 获得双向迭代器 `it` 的第 `n` 个前驱。

[STL 容器](#) 一般支持从一端或两端开始的访问，以及对 [const 修饰符](#) 的支持。例如容器的 `begin()` 函数可以获得指向容器第一个元素的迭代器，`rbegin()` 函数可以获得指向容器最后一个元素的反向迭代器，`cbegin()` 函数可以获得指向容器第一个元素的 `const` 迭代器，`end()` 函数可以获得指向容器尾端（「尾端」并不是最后一个元素，可以看作是最后一个元素的后继；「尾端」的前驱是容器里的最后一个元素，其本身不指向任何一个元素）的迭代器。

可在 [Iterator library - cppreference.com](http://iterator.library-cppreference.com) 查看更多用法。

本页面最近更新：2024/9/29 14:28:19, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[ccompute](#), [CCXXI](#), [Enter-tainer](#), [iBug](#), [lr1d](#), [Menci](#), [mgt](#), [opsiff](#), [ouuan](#), [real01bit](#), [sbofgayschool](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用